

DOCUMENTAÇÃO COMPLETA DO SIMULADOR DE CONCURSO COM IA

ÍNDICE

- Visão Geral do Projeto
 - Estrutura de Arquivos
 - Tecnologias Utilizadas
 - Funcionalidades
 - Guia de Modificações
 - Manutenção e Troubleshooting
 - Deploy
 - Referência Rápida
-

1. VISÃO GERAL DO PROJETO

Descrição

Simulador de Concurso Público que utiliza IA (DeepSeek) para gerar questões personalizadas a partir de conteúdo fornecido pelo usuário (categorias prontas ou texto personalizado).

Fluxo Principal

- Usuário seleciona uma categoria ou digita conteúdo
- Escolhe quantidade de questões (20, 40, 60, 80)
- Sistema envia para DeepSeek API
- IA gera questões com alternativas, resposta correta e explicação
- Usuário responde as questões
- Sistema mostra resultado com gabarito comentado
- Usuário pode exportar resultado em PDF

URLs

- Local: `http://localhost:3000`
 - Produção: `https://meu-simulador.vercel.app`
-

2. ESTRUTURA DE ARQUIVOS

text

```
meu-simulador/
|
├── app/
|   ├── api/
|   │   └── generate-quiz/
|   │       └── route.ts           # API que chama DeepSeek
|   │
|   ├── page.tsx                 # Página principal (Front-end)
|   ├── layout.tsx               # Layout base
|   └── globals.css              # Estilos globais + Dark Mode
|
├── public/
|   └── logo.png                 # Logo do site
|
├── tailwind.config.ts          # Configuração do Tailwind
├── package.json                # Dependências do projeto
├── .env.local                  # Variáveis de ambiente
└── next.config.js              # Configuração do Next.js
```

3. TECNOLOGIAS UTILIZADAS

Tecnologia	Versão	Finalidade
Next.js	16.2.9	Framework React com SSR
React	18.2.0	Biblioteca UI
TypeScript	5.x	Tipagem estática
Tailwind CSS	3.4.1	Estilização
DeepSeek API	-	Geração de questões com IA
jsPDF	2.5.1	Exportação de PDF
Vercel	-	Hospedagem

4. FUNCIONALIDADES

4.1. Categorias

Arquivo: `app/page.tsx` (linhas 27-133)

Como funciona:

- Array `CATEGORIAS` com nome e conteúdo pré-definido
- Opção "✎ Escrever meu próprio conteúdo" permite entrada personalizada
- Ao selecionar, o conteúdo é enviado para a API

Como adicionar uma nova categoria:

```
tsx

// Localize o array CATEGORIAS no page.tsx
const CATEGORIAS = [
  // ... categorias existentes ...
  {
    nome: '📁 Nova Categoria', // Nome com emoji
    conteudo: `Conteúdo da nova categoria...` // Texto do conteúdo
  }
];
```

4.2. Quantidade de Questões

Arquivo: `app/page.tsx` (linhas 148-163)

Valores disponíveis: 20, 40, 60, 80

Como alterar:

```
tsx

// Localize o array de botões
{[20, 40, 60, 80].map((num) => (
  // Para adicionar 100 questões, adicione ao array:
  {[20, 40, 60, 80, 100].map((num) => (
```

Limite máximo: A DeepSeek tem limite de tokens. 80 questões é o recomendado.

4.3. Geração de Questões (API)

Arquivo: `app/api/generate-quiz/route.ts`

Como funciona:

- 1.Recebe conteúdo e quantidade via POST
- 2.Valida se há conteúdo suficiente (50+ caracteres)
- 3.Verifica API Key da DeepSeek
- 4.Envia prompt para DeepSeek
- 5.Processa resposta JSON
- 6.Valida e retorna questões

Prompt da IA: Controla como as questões são geradas. Modifique para ajustar:

- Nível de dificuldade
 - Estilo das perguntas
 - Formato das alternativas
-

4.4. Modo Escuro (Dark Mode)

Arquivos: `app/page.tsx` (linhas 174-191) `globals.css` `tailwind.config.ts`

Como funciona:

- Estado `darkMode` controla o tema
- Salvo no `localStorage` do navegador
- Classe `dark` adicionada ao `<html>`
- Tailwind usa `dark:` para estilos condicionais

Como personalizar cores do Dark Mode:

`css`

```
/* globals.css */
html.dark .bg-white {
  @apply bg-gray-800; /* Mude para a cor desejada */
}

html.dark .text-gray-900 {
  @apply text-gray-100; /* Mude para a cor desejada */
}
```

4.6. Layout e Estilos

Arquivos: `app/page.tsx` (JSX) `globals.css`

Classes principais:

- `cardBg` Fundo dos cards (branco ou cinza escuro)
- `cardShadow` Sombra dos cards
- `inputBg` Fundo dos inputs
- `hoverBg` Fundo no hover

Como mudar cores principais:

`tsx`

```
// No page.tsx, localize a classe 'blue-600' e substitua por outra cor Tailwind
className="bg-blue-600 hover:bg-blue-700" // Mude 'blue' para 'green', 'purple', etc.
```

5. GUIA DE MODIFICAÇÕES

5.1. Como adicionar mais categorias

Arquivo: `app/page.tsx`

`tsx`

```
// Localize o array CATEGORIAS
const CATEGORIAS = [
  // ... categorias existentes ...
  {
    nome: '📁 Nova Categoria',
    conteudo: `Todo o conteúdo da nova categoria aqui...

    1. Tópico 1: Descrição detalhada
    2. Tópico 2: Descrição detalhada`
  }
];
```

5.2. Como mudar a quantidade máxima de questões

Arquivo: `app/page.tsx`

```
tsx

// Localize o array de botões
{[20, 40, 60, 80].map((num) => (
  // Mude para:
  {[20, 40, 60, 80, 100].map((num) => (
```

Também altere no `route.ts`:

```
tsx

// app/api/generate-quiz/route.ts
const quantidade = body?.quantidade || 40; // Valor padrão
// Se adicionar 100, não precisa alterar nada aqui
```

5.3. Como mudar o estilo do PDF

Arquivo: `app/page.tsx` (função `exportarPDF`)

Cores:

```
tsx

// Cores em RGB (0-255)
pdf.setTextColor(0, 150, 0); // Verde (correta)
pdf.setTextColor(200, 0, 0); // Vermelho (errada)
pdf.setTextColor(80, 80, 80); // Cinza (explicação)

// Para mudar:
pdf.setTextColor(0, 0, 255); // Azul
pdf.setTextColor(255, 165, 0); // Laranja
```

Fonte:

```
tsx

pdf.setFont('helvetica'); // Sans-serif (recomendado)
pdf.setFont('times'); // Serif (Times New Roman)
pdf.setFont('courier'); // Monoespacada
```

5.4. Como mudar o prompt da IA

Arquivo: `app/api/generate-quiz/route.ts`

`tsx`

```
const PROMPT_QUESTIONS = `
```

```
Você é um professor especialista em concursos públicos.
```

```
CRIE EXATAMENTE ${quantidade} QUESTÕES...
```

```
// Personalize aqui:
```

1. "As questões devem ser nível médio" → "As questões devem ser nível difícil"
2. "Use linguagem formal" → "Use linguagem simples e didática"
3. Adicione: "Crie 5 questões sobre cada tópico principal"

```
`;
```

5.5. Como adicionar um novo campo (ex: nível de dificuldade)

1. Adicione no front-end (page.tsx):

`tsx`

```
// Adicione o state
```

```
const [dificuldade, setDificuldade] = useState('médio');
```

```
// Adicione o seletor
```

```
<select onChange={(e) => setDificuldade(e.target.value)}>
```

```
  <option value="fácil">Fácil</option>
```

```
  <option value="médio">Médio</option>
```

```
  <option value="difícil">Difícil</option>
```

```
</select>
```

```
// Envie na requisição
```

```
body: JSON.stringify({
  conteudo: textoParaEnviar,
  quantidade: quantidade,
  dificuldade: dificuldade // ← Novo campo
})
```

2. Adicione no back-end (route.ts):

```
tsx

// Receba o campo
const dificuldade = body?.dificuldade || 'médio';

// Use no prompt
const PROMPT_QUESTIONS = `
...
Nível de dificuldade: ${dificuldade}
...
`;
```

6. MANUTENÇÃO E TROUBLESHOOTING

6.1. Erros Comuns

Erro	Causa	Solução
"API Key não configurada"	<code>.env.local</code> sem chave	Adicione <code>DEEPSEEK_API_KEY=sk-...</code>
"Erro ao gerar questões"	Conteúdo muito curto	Mínimo 50 caracteres
"PDF não gera"	jsPDF não instalado	<code>npm install jspdf@2.5.1</code>
"Dark Mode não funciona"	Tailwind não configurado	Verificar <code>tailwind.config.ts</code>
"Erro de build na Vercel"	Variável de ambiente faltando	Configurar na Vercel

6.2. Logs

Local: Abra o console do navegador (F12)

Vercel: Acesse "Deployments" → "View Logs"

6.3. Comandos Úteis

```
bash
```

```
# Rodar localmente
```

```
npm run dev
```

```
# Build para produção
```

```
npm run build
```

```
# Testar build
```

```
npm start
```

```
# Instalar dependências
```

```
npm install --legacy-peer-deps
```

```
# Atualizar dependências
```

```
npm update
```

7. DEPLOY

7.1. Deploy na Vercel

1.Faça push para o GitHub

2.Vercel detecta automaticamente

3.Configure variáveis de ambiente:

- `DEEPSEEK_API_KEY` sua chave

7.2. Atualizações

```
bash
```

```
git add .
```

```
git commit -m "Descrição das alterações"
```

```
git push
```

7.3. Domínio Personalizado

- 1.Na Vercel, vá em "Settings" → "Domains"
 - 2.Adicione seu domínio
 - 3.Configure DNS com a Vercel
-

8. REFERÊNCIA RÁPIDA

8.1. Arquivos e suas funções

Arquivo	O que faz
app/page.tsx	Interface principal
app/api/generate-quiz/route.ts	API de geração
app/globals.css	Estilos e dark mode
tailwind.config.ts	Configuração Tailwind
.env.local	Chaves de API

8.2. Variáveis de Ambiente

```
text
DEEPSEEK_API_KEY=sk-... # Chave da DeepSeek
```

8.3. Dependências

```
json
{
  "next": "16.2.9",
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "jspdf": "2.5.1"
}
```

8.4. Cores Principais

Elemento	Cor (Light)	Cor (Dark)
Fundo	bg-gray-50	bg-gray-900
Card	bg-white	bg-gray-800
Texto	text-gray-900	text-gray-100
Botão	bg-blue-600	bg-blue-500
Correta	text-green-600	text-green-400
Errada	text-red-600	text-red-400



CHECKLIST DE MANUTENÇÃO

- Verificar se a DeepSeek API está com saldo
- Manter dependências atualizadas
- Fazer backup do código no GitHub
- Testar após mudanças
- Verificar logs da Vercel

Documentação criada em: 22/06/2026

Última atualização: 22/06/2026